

## Taller: Taller sobre Patrones de Arquitectura de Microservicios

Descripción: Implementación de Service Discovery y Circuit Breaker en Microservicios

### Parte 1: Configuración y Comprensión de Service Discovery

Guía paso a paso:

1. Clonar los repositorios:
  - fx-discovery-server: [jrquinte/FutureXEurekaServer \(github.com\)](https://github.com/jrquinte/FutureXEurekaServer)
  - fx-course-service: [jrquinte/FutureXCourseApp \(github.com\)](https://github.com/jrquinte/FutureXCourseApp)
  - fx-catalog-service: [jrquinte/FutureXCourseCatalog \(github.com\)](https://github.com/jrquinte/FutureXCourseCatalog)
2. Configurar y ejecutar fx-discovery-server:
  - Abrir el proyecto en su IDE
  - Verificar la configuración en application.properties
  - Ejecutar FutureXEurekaServerApplication
3. Configurar y ejecutar fx-course-service:
  - Abrir el proyecto en su IDE
  - Verificar la configuración en application.properties
  - Ejecutar FutureXCourseAppApplication
4. Configurar y ejecutar fx-catalog-service:
  - Abrir el proyecto en su IDE
  - Verificar la configuración en application.properties
  - Ejecutar FutureXCourseCatalogApplication
5. Comprender la interacción entre servicios:
  - Observar cómo fx-catalog-service descubre fx-course-service a través de Eureka:

```
InstanceInfo instanceInfo = client.getNextServerFromEureka("fx-course-service", false);  
String courseAppURL = instanceInfo.getHomePageUrl();
```

- Notar cómo se realiza la llamada al servicio de cursos:

```
RestTemplate restTemplate = new RestTemplate();  
courses = restTemplate.getForObject(courseAppURL, String.class);
```

6. Probar los endpoints:

- Acceder a <http://localhost:8002/> para ver el catálogo
- Acceder a <http://localhost:8002/catalog> para ver los cursos
- Acceder a <http://localhost:8002/firstcourse> para ver el primer curso

7. Observar el panel de Eureka:

- Acceder a <http://localhost:8761/> para ver los servicios registrados

## Parte 2: Implementación de Circuit Breaker con Resilience4j

Guía paso a paso:

1. Agregar dependencias de Resilience4j en fx-catalog-service: Añadir en pom.xml:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-
resilience4j</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

2. Configurar Resilience4j en la aplicación: Añadir la siguiente configuración en application.yml o application.properties:

```
resilience4j.circuitbreaker:
  instances:
    courseService:
      registerHealthIndicator: true
      slidingWindowSize: 10
      minimumNumberOfCalls: 5
      permittedNumberOfCallsInHalfOpenState: 3
      automaticTransitionFromOpenToHalfOpenEnabled: true
      waitDurationInOpenState: 5s
      failureRateThreshold: 50
      eventConsumerBufferSize: 10
```

3. Implementar métodos de fallback: Crear métodos que se ejecutarán cuando falle la llamada al servicio de cursos. Ejemplo:

```
public String fallbackGetCatalog(Exception ex) {
  return "Fallback: Unable to retrieve course catalog.";
}
```

```
}
```

4. Aplicar Circuit Breaker a los métodos del controlador: Usar `@CircuitBreaker` en los métodos de `CatalogController`. Ejemplo:

```
@GetMapping("/catalog")
@CircuitBreaker(name = "courseService", fallbackMethod =
"fallbackGetCatalog")
public String getCatalog() {
    // ... código existente ...
}
```

5. Configurar Resilience4j (opcional): Para una configuración más avanzada, puedes crear una clase de configuración:

```
@Configuration
public class Resilience4JConfig {
    @Bean
    public Customizer<Resilience4JCircuitBreakerFactory> globalCustomConfiguration() {
        CircuitBreakerConfig circuitBreakerConfig = CircuitBreakerConfig.custom()
            .failureRateThreshold(50)
            .waitDurationInOpenState(Duration.ofMillis(1000))
            .slidingWindowSize(2)
            .build();
        return factory -> factory.configureDefault(id -> new Resilience4JConfigBuilder(id)
            .circuitBreakerConfig(circuitBreakerConfig)
            .build());
    }
}
```

6. Probar el Circuit Breaker:
  - Ejecutar `fx-catalog-service`
  - Acceder a los endpoints normalmente
  - Detener `fx-course-service`
  - Acceder a los endpoints de `fx-catalog-service` y observar el comportamiento del fallback
  - Reiniciar `fx-course-service` y observar cómo el Circuit Breaker se cierra de nuevo
7. Monitoreo (opcional): Si tienes tiempo, puedes agregar la dependencia de actuator y configurar endpoints de salud para monitorear el estado del Circuit Breaker:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Y en `application.properties`:

```
management.health.circuitbreakers.enabled=true
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=always
```

Esta guía actualizada utiliza Resilience4j, que es más ligero y más fácil de configurar que Hystrix. Además, es completamente compatible con Spring Boot 3 y proporciona una funcionalidad similar para implementar el patrón Circuit Breaker.

**Entregable:**

1. Código Fuente de implementación de Resilience4j
2. Presentación y demostración

**Criterios de Calificación:**

1. Comprensión del Service Discovery (40%)
  - Explicación clara de cómo fx-catalog-service descubre y se comunica con fx-course-service
  - Demostración de la funcionalidad a través de los endpoints
2. Implementación de Resilience4j (60%)
  - Correcta configuración de Resilience4j en el proyecto
  - Implementación adecuada de métodos de fallback
  - Uso apropiado de las anotaciones de Resilience4j en los métodos del controlador
  - Configuración adecuada de propiedades de Resilience4j en application.yml/properties
  - Demostración exitosa del funcionamiento del Circuit Breaker y otros patrones de resiliencia
  - Explicación clara de cómo Resilience4j mejora la resiliencia del sistema
  - Explicación clara de cómo Hystrix mejora la resiliencia del sistema